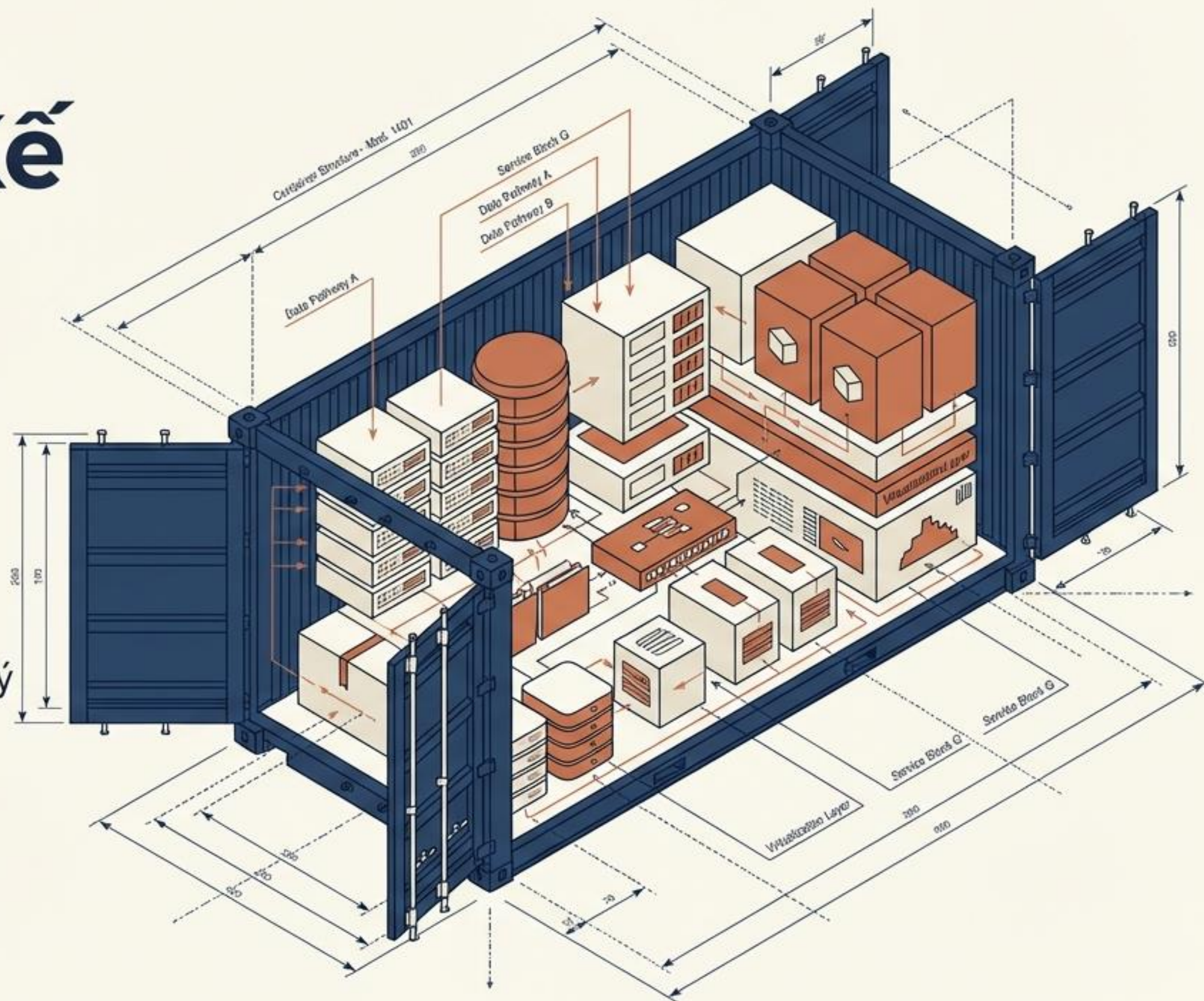
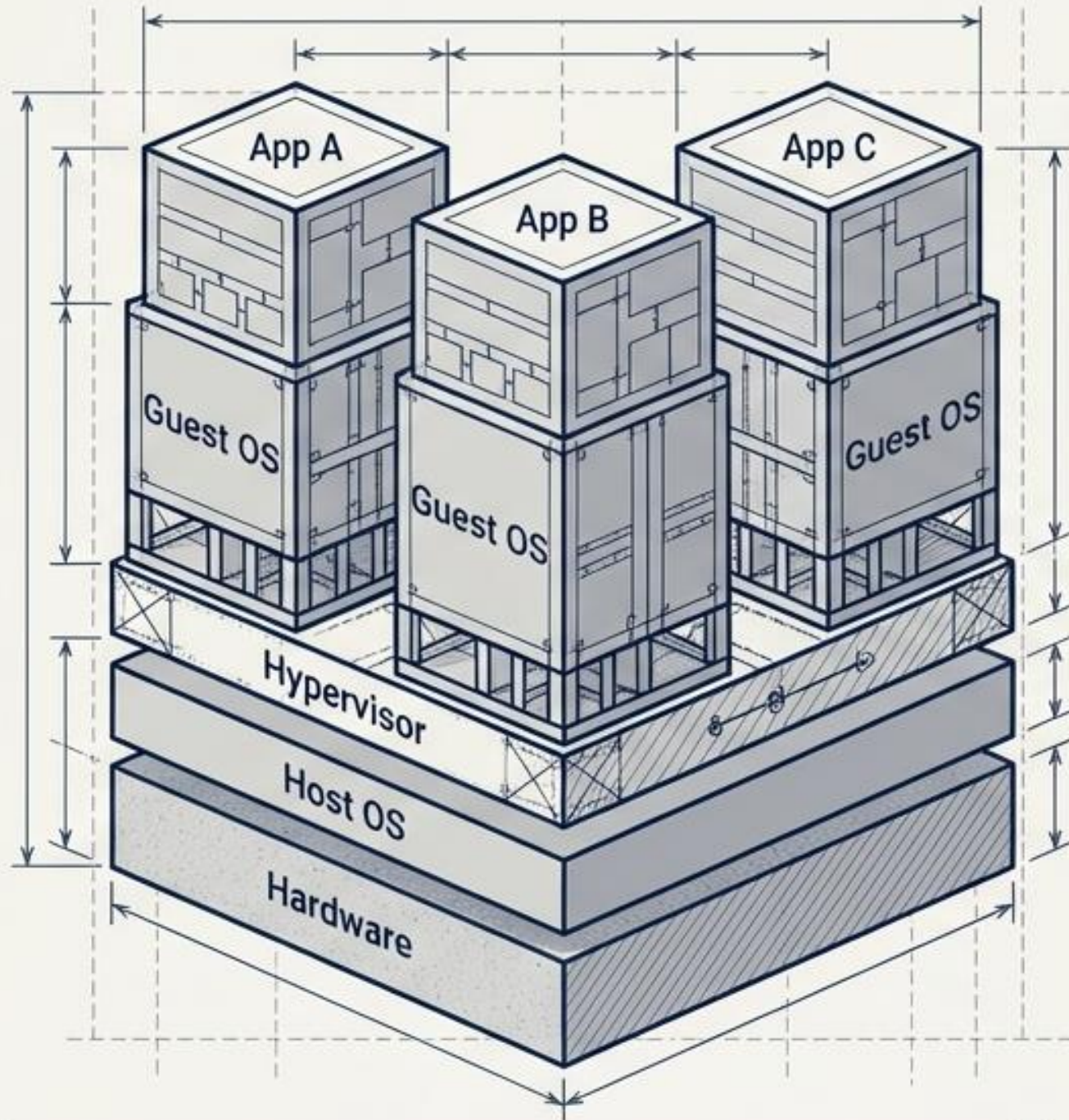


Bản Thiết Kế Docker Toàn Diện

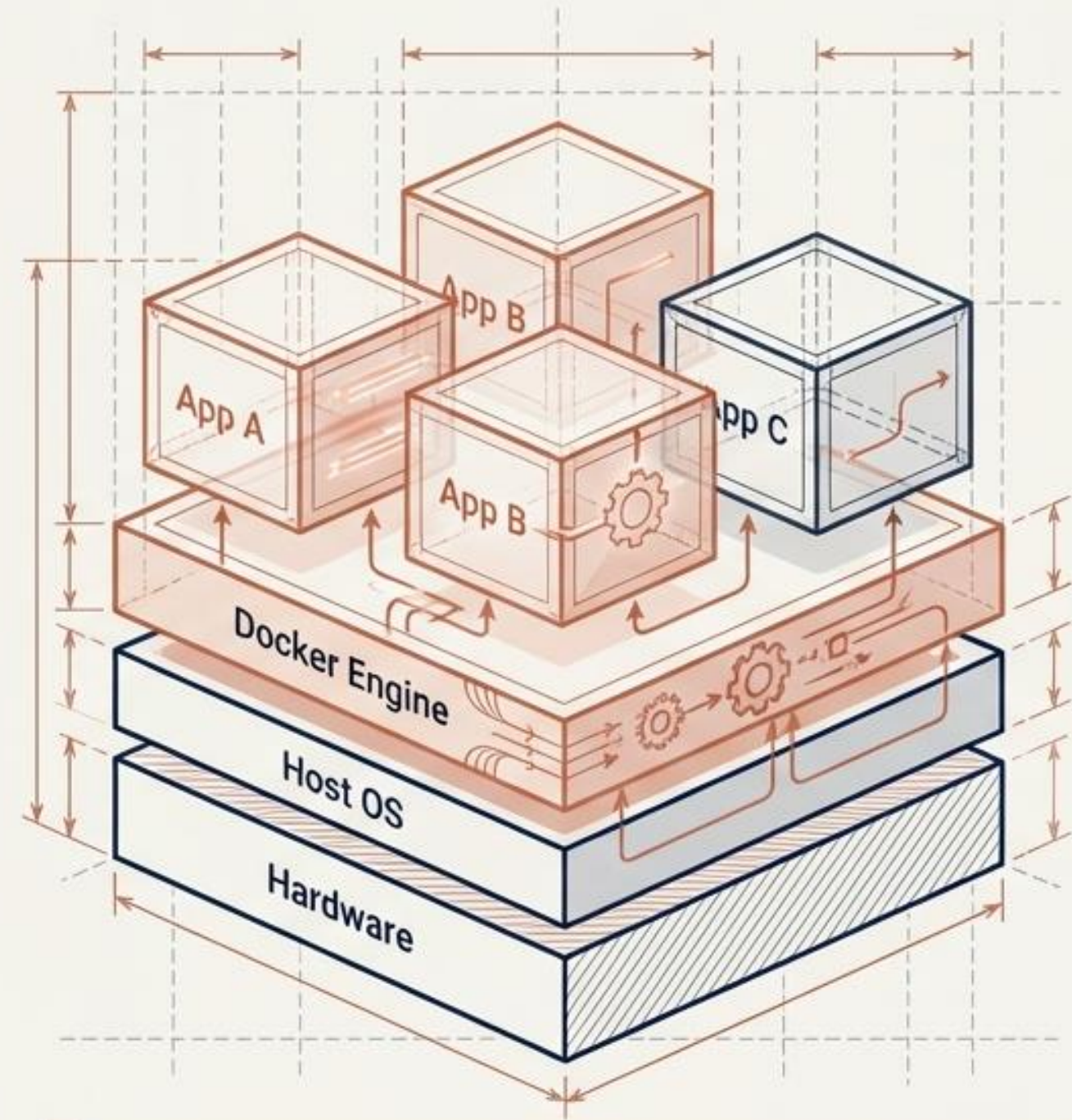
Hướng dẫn hoàn chỉnh về ảo hóa
ứng dụng và quản lý Container từ lý
thuyết đến thực tiễn.



Khái Niệm Container Hóa: Tạm Biệt Máy Chủ Ảo Công Kênh



Ảo hóa phần cứng (VMs): Yêu cầu một hệ điều hành khách (Guest OS) đầy đủ và hypervisor cho mỗi ứng dụng. Tiêu tốn nhiều tài nguyên, khởi động chậm.



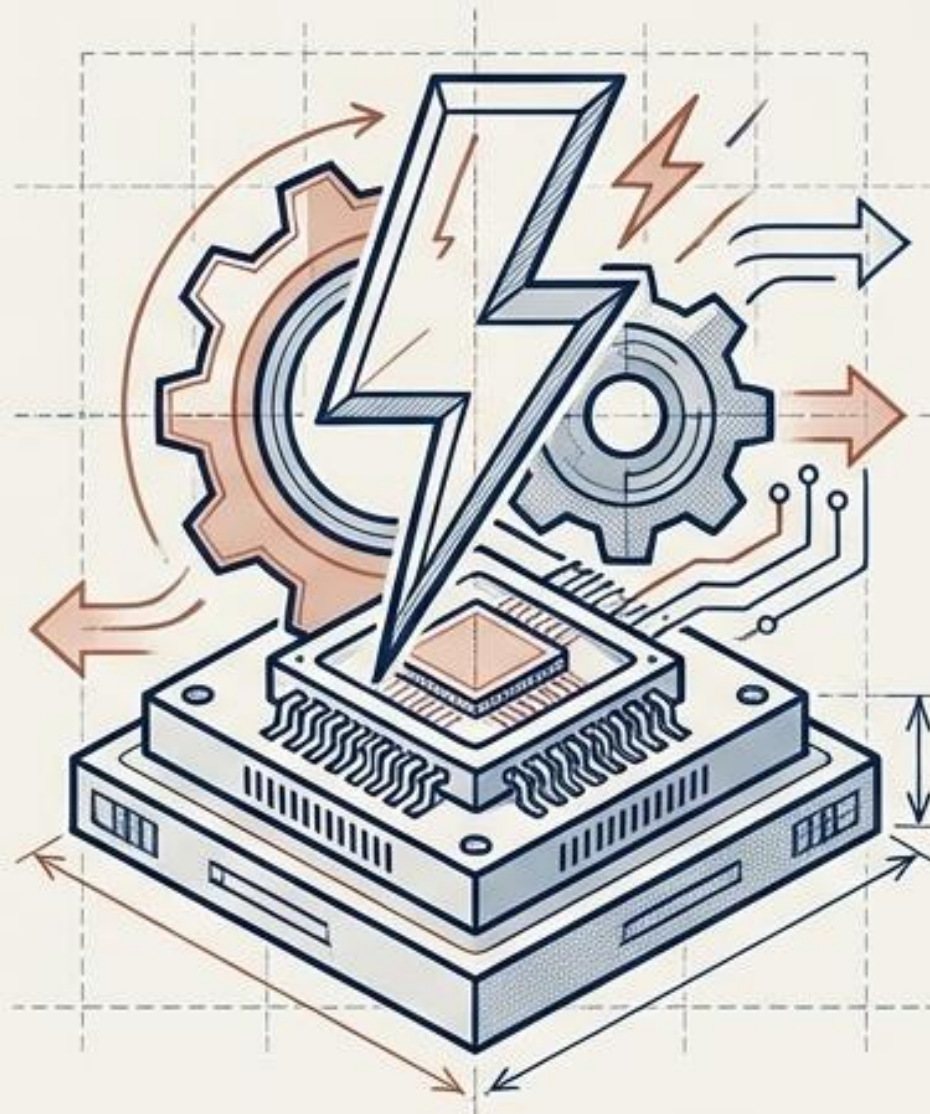
Ảo hóa cấp hệ điều hành (Containers): Đóng gói mã nguồn, thư viện và cấu hình vào một môi trường biệt lập. Chia sẻ chung Kernel của hệ điều hành máy chủ, bỏ qua Guest OS.

3 Lý Do Docker Thống Trị Vòng Đời Phát Triển Phần Mềm



Khả Năng Di Động (Portability)

Chạy đồng nhất trên mọi môi trường. Giải quyết triệt để vấn đề: "Code chạy được trên máy tôi nhưng lỗi trên server".



Tối Ưu Hiệu Suất (Efficiency)

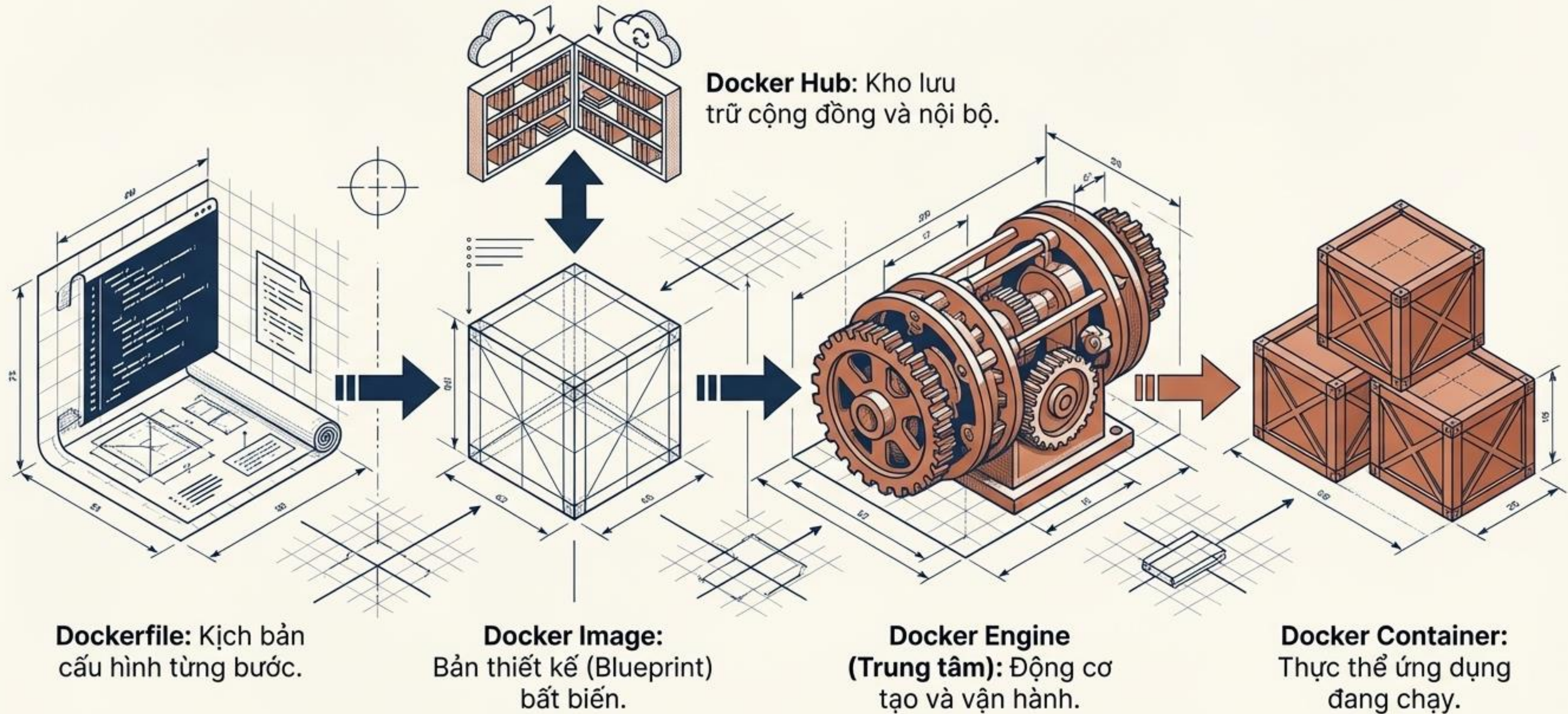
Khởi động trong tích tắc. Tiêu thụ ít CPU và RAM hơn máy ảo truyền thống, cho phép chạy mật độ cao trên cùng một máy chủ.



Sự Biệt Lập (Isolation)

Mỗi container chạy các tiến trình độc lập, với cấu hình và mạng riêng biệt. Ngăn ngừa hoàn toàn xung đột phần mềm giữa các ứng dụng.

Hệ Sinh Thái Docker: Từ Bản Thiết Kế Đến Thực Thể Hoạt Động



Kiến Trúc Client-Server Của Docker Engine

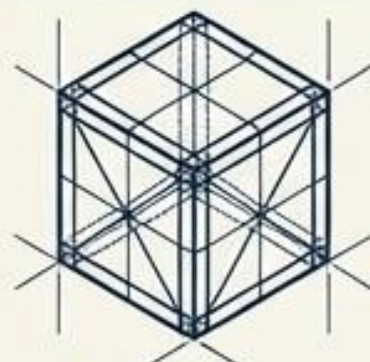


Giao diện dòng lệnh (VD: lệnh `docker run`). Nơi người dùng đưa ra “yêu cầu món ăn”.

Cầu nối giao tiếp tiêu chuẩn, truyền các chỉ thị từ Client đến Server một cách an toàn.

Tiến trình chạy ngầm (Daemon) trực tiếp đảm nhận các tác vụ nặng: tạo, chạy và quản lý vòng đời của containers.

Phân Biệt Cốt Lõi: Image vs. Container



Image



Container

Bản chất

Bản thiết kế (Blueprint) / Khuôn mẫu định sẵn

Thực thể đang chạy (Instance) được tạo ra từ khuôn mẫu.

Trạng thái

Chỉ đọc (Read-only), bất biến (Immutable)

Có thêm lớp có thể ghi (Writable layer), có vòng đời (khởi tạo, chạy, dừng, xóa).

Ví dụ

"Công thức nấu ăn" trong sách

"Món ăn hoàn chỉnh" trên bàn.

Lệnh thao tác

docker pull, docker build, docker rmi

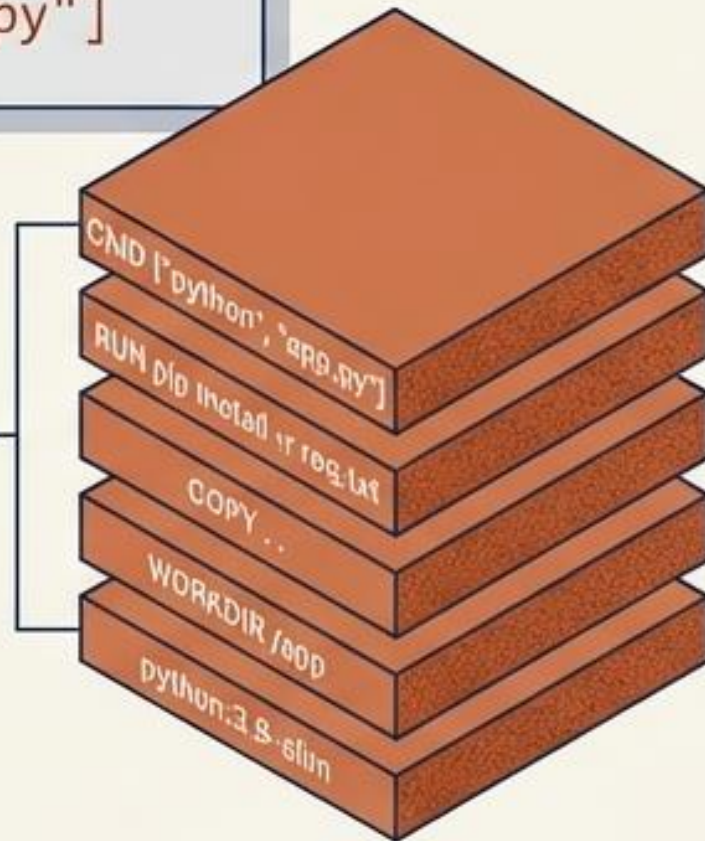
docker run, docker ps, docker rm

Tự Động Hóa Xây Dựng: Dockerfiles & Docker Compose

Tạo một Image đơn lẻ

```
FROM python:3.9-slim
WORKDIR /app
COPY . .
RUN pip install -r req.txt
CMD ["python", "app.py"]
```

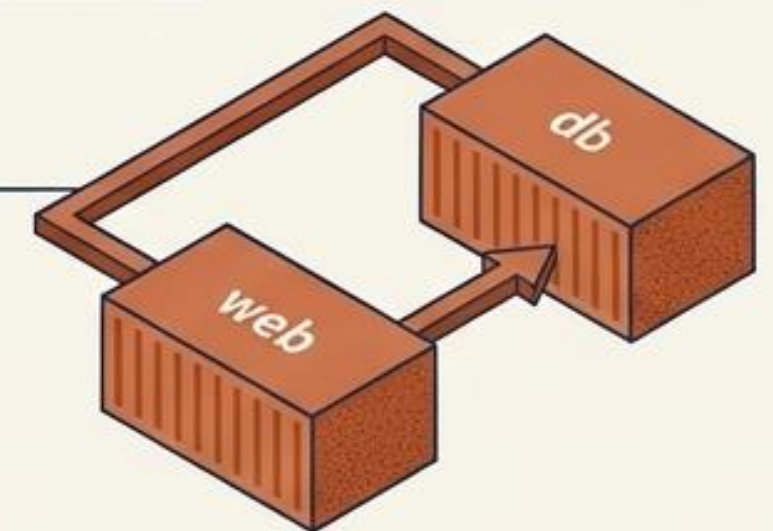
Mỗi lệnh tạo ra một "layer" (lớp) cấu trúc nên Image.



Quản lý hệ thống đa Container

```
services:
  web:
    build: .
    ports: "8000:8000"
    depends_on: [db]
  db:
    image: postgres:15
```

Định nghĩa mạng, ổ đĩa và thứ tự khởi động toàn bộ ứng dụng (docker compose up -d).

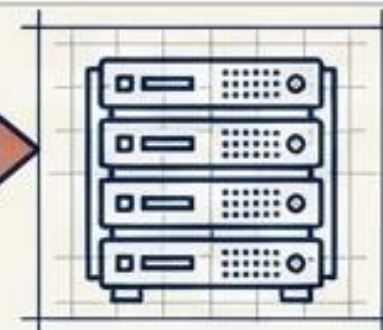


Bắt Đầu Nhanh: Cài Đặt & Các Lệnh Giao Tiếp Cơ Bản



Môi trường Desktop

Tải gói cài đặt Docker Desktop (GUI tích hợp sẵn cho Windows/Mac).



Môi trường Server

Cài đặt Docker Engine CLI qua Package Manager (APT cho Ubuntu, Homebrew cho macOS).

Terminal Cheat Sheet

<code>docker ps -a</code>	→	Liệt kê tất cả container (kể cả đã dừng).
<code>docker pull <image></code>	→	Tải bản thiết kế từ Docker Hub.
<code>docker build -t <tên> .</code>	→	Đóng gói Image mới từ Dockerfile.
<code>docker run -it <image> /bin/bash</code>	→	Khởi chạy và tương tác trực tiếp với shell của container.
<code>docker stop <id> & docker rm <id></code>	→	Dừng an toàn và xóa bỏ container.

Quản Lý Dữ Liệu: Vượt Qua Bản Chất Tạm Thời Của Container

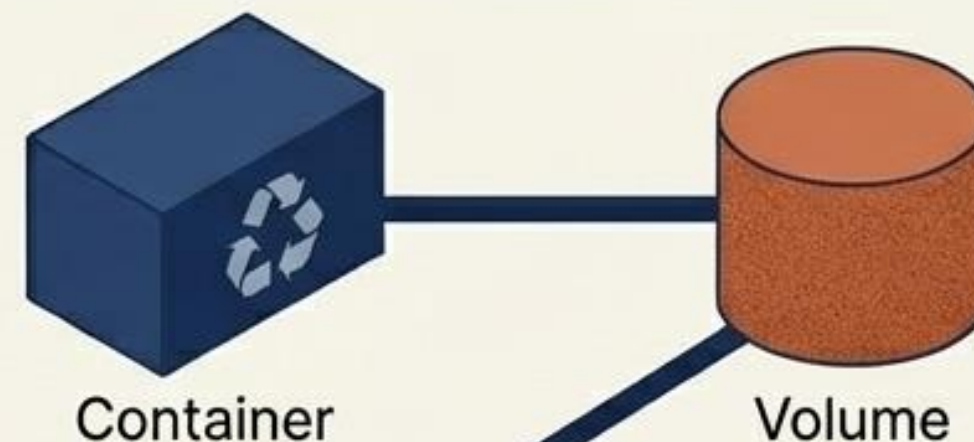
Vấn đề: Container là tạm thời (stateless). Nếu xóa container, mọi dữ liệu nội bộ bị mất vĩnh viễn.

Giải pháp - Docker Volumes: Cơ chế lưu trữ an toàn nằm ngoài container (tại `/var/lib/docker/volumes/`), được Docker quản lý trực tiếp.

Lệnh Cốt Lõi:

- Tạo mới: `docker volume create <tên_volume>`
- Gắn vào container: `docker run -v <tên_volume>:<đường_dẫn_đích>`
- Dọn dẹp không gian: `docker volume prune -a`

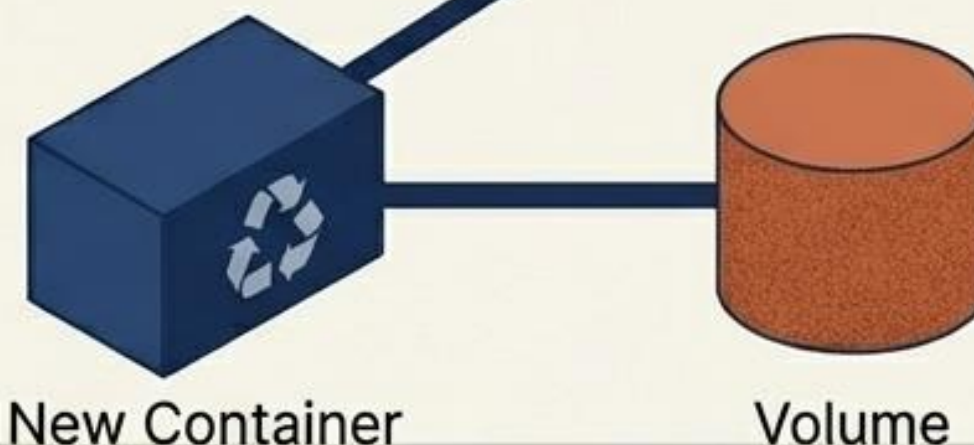
BƯỚC 1



BƯỚC 2



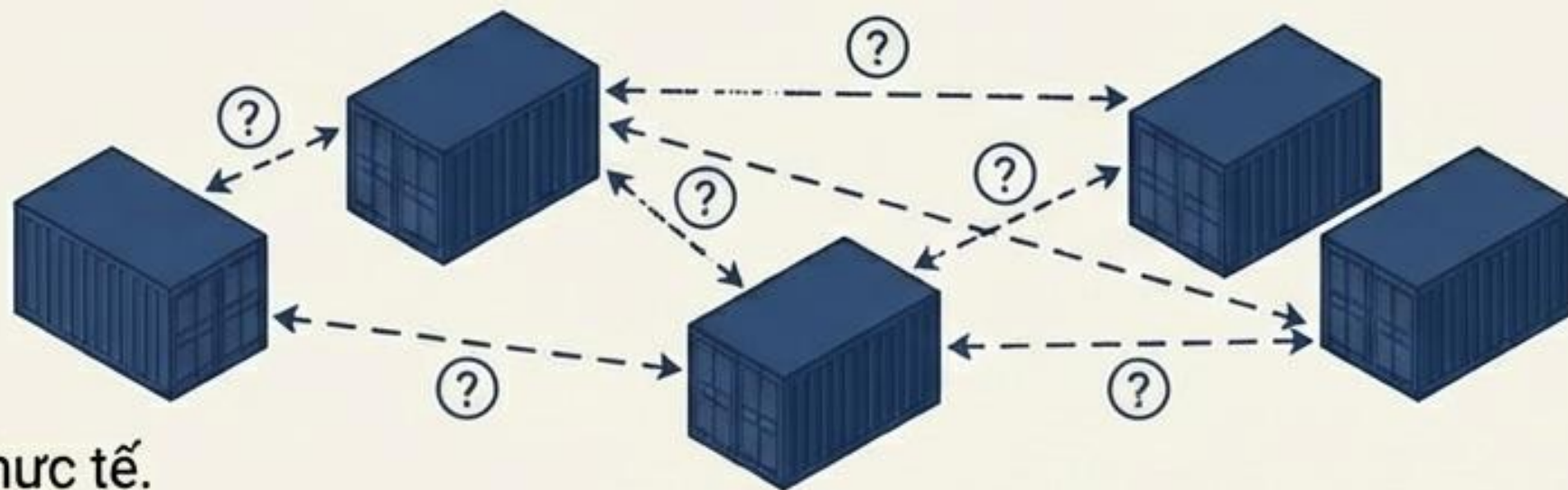
BƯỚC 3



Thiết Lập Mạng (Networking): Kết Nối Các Dịch Vụ

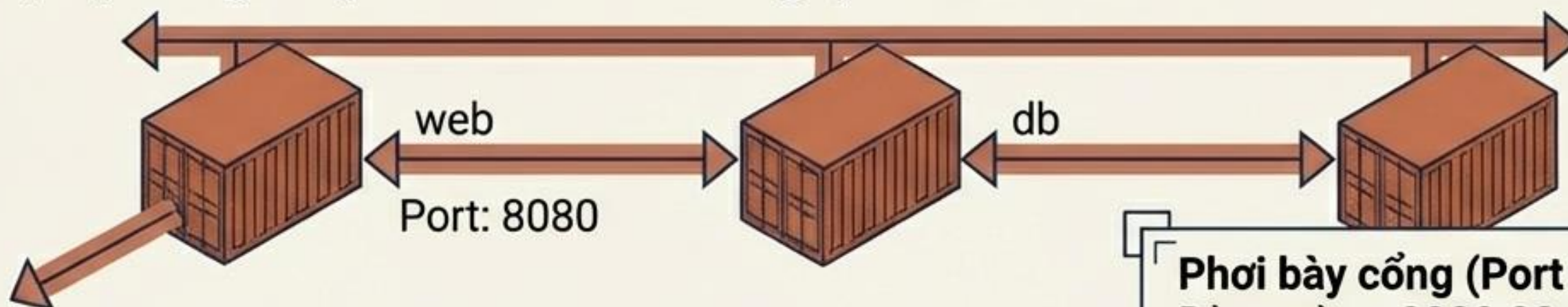
Mạng Mặc Định (Default Bridge)

Hạn chế giao tiếp giữa các container.
Phải kết nối thủ công bằng IP.
Không phù hợp cho ứng dụng đa dịch vụ thực tế.



Mạng Tự Định Nghĩa (User-Defined Bridge) - Best Practice

Cho phép các container giao tiếp tự động thông qua tên dịch vụ (**Internal DNS**). Tăng cường tính bảo mật và cách ly mạng lưới chuyên biệt (docker network create <tên_mạng>).

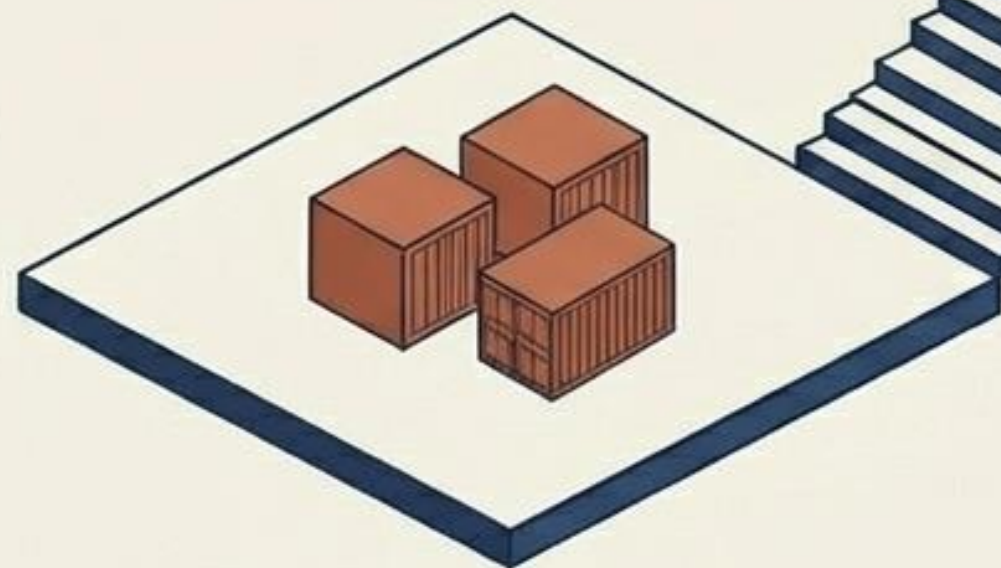


Phơi bày cổng (Port Publishing):
Dùng cờ -p 8080:80 để ánh xạ luồng truy cập từ bên ngoài vào cổng nội bộ của container.

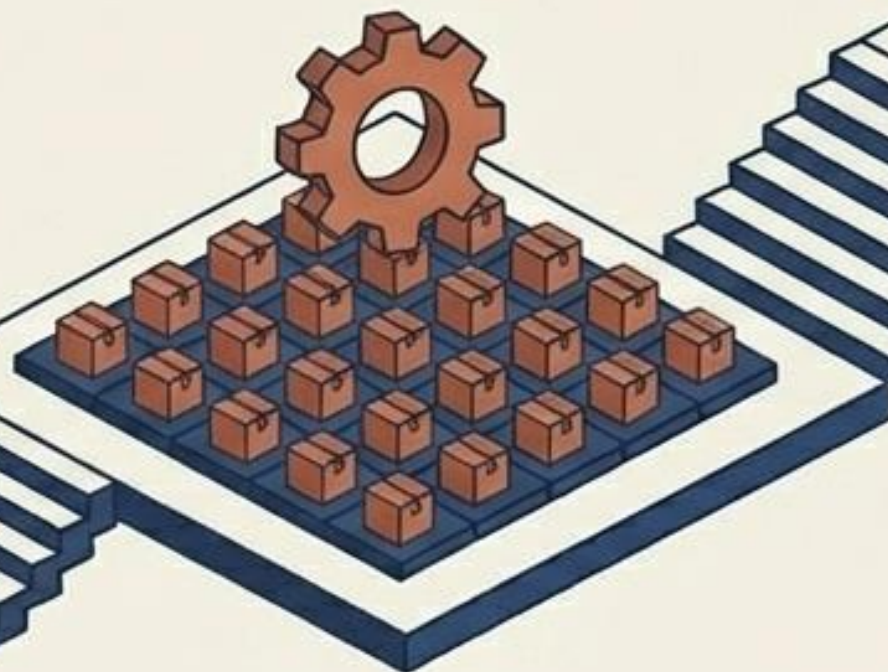
Khắc Phục Lỗi (Troubleshooting) Thường Gặp

Triệu chứng	Nguyên nhân	Cách khắc phục
Container lập tức thoát/dừng ngay sau khi chạy.	Tiến trình chính (CMD/ENTRYPOINT) đã chạy xong, hoặc thiếu cờ chạy ngầm.	Kiểm tra docker logs. Thêm cờ -d (detached mode) hoặc đảm bảo tiến trình là dịch vụ chạy liên tục.
Sửa Dockerfile nhưng Image mới không cập nhật thay đổi (lỗi package cũ).	Cơ chế bộ nhớ đệm (Layer Caching) tái sử dụng các lớp cũ để tiết kiệm thời gian.	Dùng lệnh build với cờ --no-cache. Gộp lệnh apt-get update và install vào cùng một dòng RUN.
Lỗi "denied: requested access to the resource is denied" khi push image.	CLI chưa được xác thực với Registry (như Docker Hub).	Chạy docker login để xác thực, đảm bảo đã tag image chuẩn xác với định dạng <host>/<repo>/<tên>:<tag>.

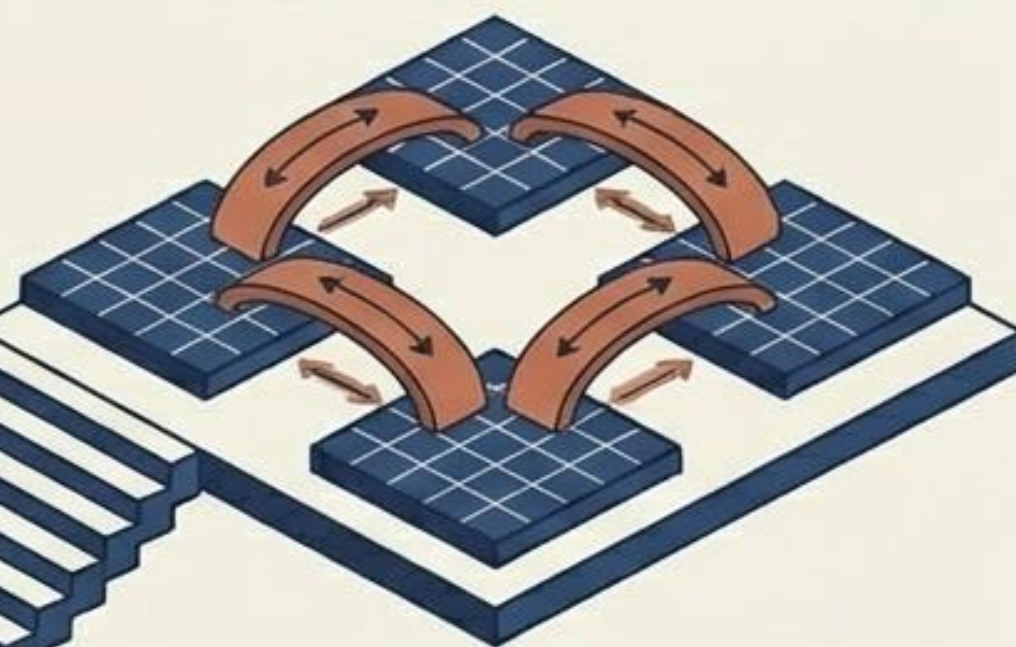
Bước Tiếp Theo: Từ Máy Trạm Đến Cụm Máy Chủ



Làm chủ Docker Compose Nâng Cao:
Kiểm soát môi trường đa dịch vụ phức tạp, định nghĩa network và volume nội bộ hoàn chỉnh chỉ bằng một tệp cấu hình YAML.



Điều Phối Hệ Thống (Orchestration):
Chuyển dịch từ việc chạy container đơn lẻ sang quản lý hàng ngàn container trên cụm máy chủ với Kubernetes hoặc Docker Swarm.



Mạng Xuyên Nút (Overlay Networks): Triển khai kiến trúc mạng phân tán, cho phép các container giao tiếp bảo mật xuyên suốt nhiều máy chủ vật lý khác nhau.

Công cụ đã sẵn sàng. Hãy bắt đầu xây dựng, đóng gói và triển khai ứng dụng của bạn ngay hôm nay.